

# 动作捕捉AI远程体育训练评价\_小白实施步骤\_Obsidian

## 模块1：动作采集

你要学会用 FZMotion 或老师提供的动捕系统采集动作。  
你不需要自己发明动作捕捉算法，重点是会做这些事：

布置摄像机完成摄像机标定贴39个标记点做T-Pose绑定骨骼采集标准动作和学生动作导出CSV和FBX文件

CSV 是给 Python 算法用的。  
FBX 是给 Unity 动画回放用的。

## 模块2：Python动作分析

你要用 Python 读取 CSV 文件，计算学生动作和标准动作的差异。  
你主要做四类指标：

ROM：动作幅度DTW：动作节奏一致性对称性：左右身体动作是否协调RMSE：关键关节角度误差

最后把这几个指标合成一个 0-100 分的综合评分。

## 模块3：Unity虚拟仿真

你要在 Unity 里做一个“双通道对比回放”。  
意思是：

左边显示标准动作右边显示学生动作两个人同时播放如果某个关节偏差太大，就高亮显示

你不一定要做得很炫，重点是能看出：

学生动作哪里和标准动作不一样哪个关节偏差最大关键姿态在哪一帧出错

## 模块4：远程网络闭环

你要把系统串起来：

采集端上传数据服务器端或AI服务端计算评分Unity端显示结果报告回传给用户

如果学校要求用 CENI，你就把它理解成一个“未来网络实验平台”。你要证明你的数据不是只在本机跑，而是经历了远程传输。

你需要记录：

上传Task ID端到端延迟网络丢包情况Wireshark/tcpdump抓包文件

## 模块5：故障诊断

老师会故意制造问题，例如：

遮挡几个动捕标记点加入100-500ms网络延迟加入1%随机丢包让AI评分漂移让Unity帧同步错位

你要证明你能定位问题。

定位方法包括：

看FZMotion日志看Python异常检测结果看Wireshark抓包看Unity帧同步日志对比优化前后评分结果

## 基于动作捕捉与AI分析的远程体育动作训练评价应用案例：小白实施步骤

核心思路：先跑通本地闭环，再接远程网络，再做故障诊断。

本地最小闭环：

CSV → Python评分 → JSON结果 → PDF报告 → Unity播放FBX

远程闭环：

上传CSV/FBX → AI服务评分 → 回传JSON/PDF → Unity显示

故障诊断：

制造异常 → 定位问题 → 修复 → 对比优化前后

# 0. 项目目录准备

建议先建一个总文件夹：

```
MotionTrainingProject/
├── data/
│   ├── raw_csv/           原始CSV
│   ├── raw_fbx/          原始FBX
│   ├── processed_csv/     预处理后的CSV
│   └── reports/           PDF报告
├── python_ai/
│   ├── read_csv.py
│   ├── preprocess.py
│   ├── metrics.py
│   ├── score.py
│   └── generate_report.py
├── unity_project/         Unity工程
├── network_service/
│   ├── main.py            FastAPI服务
│   └── uploads/
├── logs/
│   ├── fzmotion/
│   ├── python/
│   ├── unity/
│   └── network/
└── docs/
    ├── 数据字典.md
    ├── 接口说明.md
    ├── 测试报告.md
    └── 故障诊断报告.md
```

Git 初始化：

```
git init
git add .
git commit -m "init project structure"
```

## 1. 模块1：动作采集怎么做

### 1.1 你的目标

完成这条链路：

布置摄像机 → 标定 → 贴点 → T-Pose → 绑定骨骼 → 采集动作 → 导出CSV/FBX

## 1.2 你要得到什么文件

至少要有这四个文件：

|                              |         |
|------------------------------|---------|
| STD-basketball-shot-001.csv  | 标准动作CSV |
| STD-basketball-shot-001.fbx  | 标准动作FBX |
| S001-basketball-shot-001.csv | 学生动作CSV |
| S001-basketball-shot-001.fbx | 学生动作FBX |

小白建议先选 **篮球投篮**，不要一开始选舞蹈或体操。

篮球投篮的关键关节比较清楚：

右肩  
右肘  
右腕  
髋关节  
膝关节  
踝关节

---

## 1.3 步骤1：布置摄像机

你要做的不是研究摄像机原理，而是保证动作区域能被完整看见。

需要检查：

1. 人站在采集区域中央。
2. 摄像机能覆盖全身。
3. 手臂上举、下蹲、转身时，标记点不能大面积丢失。
4. 场地光照稳定，避免反光。
5. 标记点不要被衣服遮挡。

采集前拍一张场地照片，放到报告里。

证据文件：

docs/采集空间布置图.png

docs/摄像机布置照片.png

## 1.4 步骤2：摄像机标定

在 FZMotion 或老师提供的动捕软件里，一般会有 **Calibration / 标定** 功能。

你要记录：

标定时间

摄像机数量

采集空间大小

重投影误差

是否小于**0.5px**

验收标准：

| 标定结果                           | 评价         |
|--------------------------------|------------|
| 重投影误差 $\leq 0.5\text{px}$      | 优秀         |
| $0.5\text{px} \sim 1\text{px}$ | 勉强可用       |
| $> 1\text{px}$                 | 不合格，需要重新标定 |

报告里可以写：

本组完成光学动作捕捉系统标定，标定后平均重投影误差为**0.43px**，满足实验要求中 **$\leq 0.5\text{px}$** 的采集精度指标。

证据文件：

logs/fzmotion/calibration\_report.pdf

docs/标定结果截图.png

## 1.5 步骤3：贴39个标记点

你不用自己设计标记点方案，按老师或 FZMotion 的人体模板贴。

注意事项：

1. 左右不要贴反。
2. 肩、肘、腕、髌、膝、踝这些关键点要贴牢。
3. 贴点后先让受试者做几个动作，观察是否掉点。
4. 采集前拍摄39点位布设照片。

证据文件：

docs/39点位布设照片.png

---

## 1.6 步骤4：T-Pose 和骨骼绑定

T-Pose 是让人双臂水平展开，系统用这个姿势建立人体骨架。

操作步骤：

1. 让受试者站直。
2. 双臂水平展开。
3. 手心朝下或按系统要求。
4. 双脚自然分开。
5. 在软件里执行 T-Pose / Characterize / Skeleton Binding。

检查是否成功：

1. 虚拟骨骼没有扭曲。
2. 左右肩高度基本一致。
3. 手臂没有反向。
4. 腿部骨骼没有交叉。

| T-Pose 很关键。如果 T-Pose 错了，后面所有关节角度都会偏。

证据文件：

docs/T-Pose截图.png  
logs/fzmotion/skeleton\_binding\_log.txt

---

## 1.7 步骤5：采集标准动作和学生动作

## 建议采集：

标准动作：3组  
学生动作：3组  
每组动作持续：3~5秒  
采集帧率：100fps

## 命名规范：

STD-basketball-shot-001.csv  
STD-basketball-shot-002.csv  
STD-basketball-shot-003.csv  
  
S001-basketball-shot-001.csv  
S001-basketball-shot-002.csv  
S001-basketball-shot-003.csv

## 每组动作都要记录：

受试者ID  
动作类型  
采集序号  
采集时间  
是否有遮挡  
是否有缺帧  
是否重采

## 记录表示例：

| 文件名                          | 类型   | 是否有效 | 问题     |
|------------------------------|------|------|--------|
| STD-basketball-shot-001.csv  | 标准动作 | 有效   | 无明显遮挡  |
| S001-basketball-shot-001.csv | 学生动作 | 有效   | 右腕短暂缺帧 |

## 1.8 步骤6：导出CSV和FBX

CSV 用于 Python 计算。

FBX 用于 Unity 动画回放。

导出后检查：

CSV是否能用Excel打开  
CSV是否有frame/time/joint/x/y/z字段  
CSV帧率是否约为100fps  
FBX是否能导入Unity  
FBX是否能正常播放

模块1最终交付物：

1. 标定报告
2. 39点位照片
3. T-Pose截图
4. 标准动作CSV/FBX
5. 学生动作CSV/FBX
6. 数据命名说明
7. Git标签：v1.0采集规范

Git 操作：

```
git add .  
git commit -m "v1.0 motion capture data and calibration"  
git tag v1.0
```

---

## 2. 模块2：Python动作分析怎么做

### 2.1 你的目标

把 CSV 变成评分结果。

读取CSV → 清洗数据 → 计算关节角度 → 计算ROM/DTW/对称性/RMSE → 输出总分

---

### 2.2 第一步：先读懂CSV

先不要直接写评分算法。先写一个 `read_csv.py`，看看 CSV 里面有什么。

```
import pandas as pd
```

```
csv_path = "../data/raw_csv/S001-basketball-shot-001.csv"

df = pd.read_csv(csv_path)

print(df.head())
print(df.columns)
print(df.shape)
print(df.isnull().sum())
```

你要回答这几个问题：

1. 一共有多少帧？
2. 一共有多少个关节？
3. x/y/z坐标列叫什么？
4. 时间列叫什么？
5. 有没有缺失值？
6. 帧率是不是100fps？

如果连 CSV 字段都没搞清楚，后面的算法不要急着写。

## 2.3 第二步：写数据字典

在 docs/数据字典.md 里写清楚：

```
frame: 帧编号
time: 时间戳，单位s
joint_name: 关节名称
x: 关节X坐标
y: 关节Y坐标
z: 关节Z坐标
```

如果你的 CSV 是宽表，例如：

```
frame, time, right_elbow_x, right_elbow_y, right_elbow_z
```

也要写清楚每一列代表什么。

## 2.4 第三步：缺帧检测

缺帧就是某些帧里关节坐标没了。

检查函数：

```
import pandas as pd

def check_missing(csv_path):
    df = pd.read_csv(csv_path)
    total_values = df.size
    missing_values = df.isnull().sum().sum()
    missing_ratio = missing_values / total_values

    print("总数据量:", total_values)
    print("缺失值数量:", missing_values)
    print("缺失比例:", missing_ratio)

    return missing_ratio

check_missing("../data/raw_csv/S001-basketball-shot-001.csv")
```

验收指标：

缺帧插补后异常帧占比 < 1%

## 2.5 第四步：做插值补帧

如果某个关节短暂缺失，可以用线性插值。

```
import pandas as pd

def interpolate_missing(input_path, output_path):
    df = pd.read_csv(input_path)

    numeric_cols = df.select_dtypes(include=["float64", "int64"]).columns
    df[numeric_cols] = df[numeric_cols].interpolate(method="linear")
    df[numeric_cols] =
df[numeric_cols].fillna(method="bfill").fillna(method="ffill")

    df.to_csv(output_path, index=False)

interpolate_missing(
```

```

    "../data/raw_csv/S001-basketball-shot-001.csv",
    "../data/processed_csv/S001-basketball-shot-001-clean.csv"
)

```

作用：

让短暂缺失的数据变得连续，避免评分程序崩溃。

## 2.6 第五步：高斯滤波

动捕数据可能抖动，需要滤波。

```

import pandas as pd
from scipy.ndimage import gaussian_filter1d

def gaussian_smooth(input_path, output_path, sigma=2):
    df = pd.read_csv(input_path)
    numeric_cols = df.select_dtypes(include=["float64", "int64"]).columns

    for col in numeric_cols:
        if col not in ["frame", "time"]:
            df[col] = gaussian_filter1d(df[col], sigma=sigma)

    df.to_csv(output_path, index=False)

```

作用：

减少关节坐标抖动，让角度曲线更平滑。

## 2.7 第六步：计算关节角度

关节角度一般由三个点决定。

例如右肘角度需要：

右肩点 A  
 右肘点 B  
 右腕点 C

角度就是：

∠ABC

Python 函数：

```
import numpy as np

def calculate_angle(a, b, c):
    # a, b, c 分别是三个点的三维坐标
    # 返回 b 点处的夹角，单位：度
    a = np.array(a)
    b = np.array(b)
    c = np.array(c)

    ba = a - b
    bc = c - b

    cos_angle = np.dot(ba, bc) / (np.linalg.norm(ba) * np.linalg.norm(bc))
    cos_angle = np.clip(cos_angle, -1.0, 1.0)

    angle = np.degrees(np.arccos(cos_angle))
    return angle
```

先从一个关节开始做，比如右肘。

不要一上来算全身。

## 2.8 第七步：画标准动作和学生动作曲线

例如画右肘角度：

```
import matplotlib.pyplot as plt

plt.plot(std_elbow_angles, label="Standard Right Elbow")
plt.plot(stu_elbow_angles, label="Student Right Elbow")
plt.xlabel("Frame")
plt.ylabel("Angle Degree")
plt.legend()
plt.savefig("../data/reports/right_elbow_angle_compare.png")
plt.show()
```

这张图非常重要，报告和答辩都能用。

解释方式：

横轴是帧编号。  
纵轴是右肘角度。  
蓝线是标准动作。  
橙线是学生动作。  
两条线差距越大，说明动作偏差越明显。

## 3. 四个评价指标具体怎么做

### 3.1 指标1：ROM 动作幅度

ROM 是某个关节角度的最大值减最小值。

```
def calculate_rom(angle_sequence):  
    return max(angle_sequence) - min(angle_sequence)
```

例如：

标准动作右肘ROM =  $110^{\circ}$   
学生动作右肘ROM =  $92^{\circ}$   
差异 =  $18^{\circ}$

说明：

学生右肘伸展幅度不足。

ROM 分数可以简单这样算：

```
def rom_score(std_rom, stu_rom):  
    diff = abs(std_rom - stu_rom)  
    score = max(0, 100 - diff * 2)  
    return score
```

如果 ROM 差  $10^{\circ}$ ，分数就是 80 分。

## 3.2 指标2：DTW 节奏一致性

DTW 用来比较两段动作曲线的节奏是否一致。

安装：

```
pip install fastdtw
```

代码：

```
from fastdtw import fastdtw
from scipy.spatial.distance import euclidean
import numpy as np

def dtw_score(std_seq, stu_seq):
    std_seq = np.array(std_seq).reshape(-1, 1)
    stu_seq = np.array(stu_seq).reshape(-1, 1)

    distance, path = fastdtw(std_seq, stu_seq, dist=euclidean)

    score = max(0, 100 - distance / len(path))
    return score, distance
```

理解：

DTW距离越小，动作节奏越接近。  
DTW距离越大，动作节奏差异越明显。

---

## 3.3 指标3：左右对称性

对称性就是比较左侧和右侧关节动作是否接近。

例如比较：

左膝角度曲线  
右膝角度曲线

简单做法：

```
import numpy as np

def symmetry_score(left_angles, right_angles):
    left = np.array(left_angles)
    right = np.array(right_angles)

    min_len = min(len(left), len(right))
    left = left[:min_len]
    right = right[:min_len]

    diff = np.mean(np.abs(left - right))
    score = max(0, 100 - diff * 2)
    return score
```

评价解释：

分数  $\geq 85$ ：左右协调性较好  
 70~85：存在轻微不协调  
 <70：左右差异明显

## 3.4 指标4：RMSE 关键关节角度误差

RMSE 用来计算学生动作和标准动作的平均角度误差。

```
import numpy as np

def rmse_score(std_angles, stu_angles):
    std = np.array(std_angles)
    stu = np.array(stu_angles)

    min_len = min(len(std), len(stu))
    std = std[:min_len]
    stu = stu[:min_len]

    rmse = np.sqrt(np.mean((std - stu) ** 2))
    score = max(0, 100 - rmse * 2)

    return score, rmse
```

解释：

RMSE越小，学生动作越接近标准动作。

RMSE越大，说明关键关节偏差越明显。

实验要求中：

关键帧角度RMSE计算误差  $< 3^\circ$

这个意思不是学生和标准动作只差  $3^\circ$ ，而是你算法计算角度本身的误差要尽量小。

## 4. 综合评分怎么做

先用固定权重，不要一开始搞复杂模型。

```
def final_score(rom, dtw, symmetry, rmse):
    score = (
        0.25 * rom +
        0.25 * dtw +
        0.20 * symmetry +
        0.30 * rmse
    )
    return round(score, 2)
```

输出结果建议保存成 JSON：

```
{
  "student_id": "S001",
  "action_type": "basketball_shot",
  "total_score": 78.5,
  "rom_score": 82.0,
  "dtw_score": 75.0,
  "symmetry_score": 80.0,
  "rmse_score": 76.0,
  "error_joints": [
    {
      "joint": "right_elbow",
      "error_degree": 18.6,
      "frame": 125,
      "suggestion": "右肘伸展不足，建议加强出手阶段肘关节上举和伸展。"
    }
  ]
}
```

```
]
}
```

这个 JSON 后面要给 Unity 和网络服务用。

## 5. 生成PDF反馈报告怎么做

小白版本先用 `matplotlib` 保存图，再用 `reportlab` 生成 PDF。

安装：

```
pip install reportlab matplotlib
```

报告内容至少包括：

学生ID  
动作类型  
综合评分  
ROM/DTW/对称性/RMSE四项得分  
关键关节误差  
动作曲线图  
主要问题  
改进建议

示例结论：

综合评分：**78.5分**

主要问题：

1. 右肘出手阶段伸展不足，最大偏差**18.6°**。
2. 右膝屈伸幅度偏小，影响下肢发力。
3. 学生动作节奏较标准动作滞后约**0.3秒**。

改进建议：

1. 出手前保持肘关节充分上举。
2. 加强下肢蹬伸与上肢出手同步。
3. 使用分解动作训练投篮节奏。

模块2最终交付物：

1. Python读取CSV脚本
2. 数据预处理脚本
3. ROM/DTW/对称性/RMSE计算脚本
4. JSON评分结果
5. 折线图、雷达图
6. PDF反馈报告
7. Git标签: v2.0评分服务

Git:

```
git add .  
git commit -m "v2.0 python scoring service"  
git tag v2.0
```

---

## 6. 模块3: Unity虚拟仿真怎么做

### 6.1 你的目标

做一个画面:

左边: 标准动作人物  
右边: 学生动作人物  
两边同时播放  
偏差大的关节高亮  
旁边显示评分结果

---

### 6.2 第一步: 创建Unity工程

建议版本:

Unity 2021 LTS 或 Unity 2022 LTS  
模板选择: 3D Core

工程结构建议:

Assets/  
└─ Animations/

```
├─ FBX/  
├─ Scripts/  
├─ Materials/  
├─ UI/  
└─ Results/
```

## 6.3 第二步：导入FBX

把文件拖进去：

```
Assets/FBX/STD-basketball-shot-001.fbx  
Assets/FBX/S001-basketball-shot-001.fbx
```

选中 FBX 后检查：

```
Rig → Animation Type → Humanoid  
Avatar Definition → Create From This Model
```

然后点击：

```
Apply
```

如果是非标准骨骼，也可以用 Generic，但 Humanoid 更适合人体动作。

## 6.4 第三步：场景中放两个人物

在场景中放两个模型：

```
StandardAvatar: 位置 x = -1.5  
StudentAvatar: 位置 x = 1.5
```

左边是标准动作，右边是学生动作。

可以加两个文字标签：

```
标准动作  
学生动作
```

## 6.5 第四步：让两个动作同时播放

给两个模型都加 Animator。

创建两个 Animator Controller：

```
Standard.controller  
Student.controller
```

分别绑定对应动作。

小白版本可以先手动播放，进阶再用脚本控制。

需要做到：

1. 点击**Play**，两个动作同时开始。
2. 两个动作能暂停。
3. 两个动作能重新播放。
4. 两边帧同步误差尽量 $\leq 2$ 帧。

## 6.6 第五步：Unity读取JSON评分结果

把 Python 生成的 JSON 放到：

```
Assets/Results/result.json
```

Unity 里写一个简单的类：

```
[System.Serializable]  
public class ScoreResult  
{  
    public string student_id;  
    public string action_type;  
    public float total_score;  
    public float rom_score;  
    public float dtw_score;  
    public float symmetry_score;  
    public float rmse_score;  
    public ErrorJoint[] error_joints;  
}
```

```
[System.Serializable]
public class ErrorJoint
{
    public string joint;
    public float error_degree;
    public int frame;
    public string suggestion;
}
```

读取 JSON:

```
using UnityEngine;
using System.IO;

public class LoadScoreJson : MonoBehaviour
{
    public string fileName = "result.json";

    void Start()
    {
        string path = Path.Combine(Application.dataPath, "Results",
fileName);
        string json = File.ReadAllText(path);

        ScoreResult result = JsonUtility.FromJson<ScoreResult>(json);

        Debug.Log("学生ID: " + result.student_id);
        Debug.Log("总分: " + result.total_score);
    }
}
```

## 6.7 第六步：做关节高亮

最简单方式：

在出错关节附近放一个红色小球。

例如 JSON 里写：

```
{  
  "joint": "right_elbow",  
  "error_degree": 18.6,  
  "frame": 125  
}
```

Unity 里找到右肘骨骼节点，然后在它的位置生成红色小球。

不需要一开始做复杂特效。只要能说明：

右肘偏差超过 $15^\circ$ ，所以系统将右肘高亮。

高亮规则：

误差  $\leq 10^\circ$ ：不高亮

$10^\circ < \text{误差} \leq 15^\circ$ ：黄色提示

误差  $> 15^\circ$ ：红色高亮

---

## 6.8 第七步：关键姿态截图

需要保存几张图：

1. 标准动作与学生动作同时播放截图
2. 右肘高亮截图
3. 关键帧姿态对比截图
4. 评分面板截图

这些图是最终报告的证据。

模块3最终交付物：

1. Unity工程
2. 双通道回放截图
3. 偏差关节高亮截图
4. 关键姿态截图
5. 3分钟闭环演示视频
6. Git标签：v3.0闭环达标

Git：

```
git add .  
git commit -m "v3.0 unity dual-channel playback"  
git tag v3.0
```

---

## 7. 模块4：远程网络闭环怎么做

### 7.1 你的目标

把本地 Python 评分程序封装成一个网络服务。

闭环变成：

```
客户端上传CSV  
↓  
FastAPI服务接收CSV  
↓  
调用Python评分算法  
↓  
生成JSON和PDF  
↓  
返回结果  
↓  
Unity显示评分
```

先用本地 FastAPI 模拟，后面再接学校的 CENI。

---

### 7.2 第一步：安装FastAPI

```
pip install fastapi uvicorn python-multipart
```

---

### 7.3 第二步：写上传接口

在 `network_service/main.py` 里写：

```
from fastapi import FastAPI, UploadFile, File  
import shutil
```

```

import time
import uuid
import os

app = FastAPI()

UPLOAD_DIR = "uploads"
os.makedirs(UPLOAD_DIR, exist_ok=True)

@app.post("/upload_motion")
async def upload_motion(
    standard_csv: UploadFile = File(...),
    student_csv: UploadFile = File(...)
):
    start_time = time.time()

    task_id = str(uuid.uuid4())

    std_path = os.path.join(UPLOAD_DIR, f"{task_id}_standard.csv")
    stu_path = os.path.join(UPLOAD_DIR, f"{task_id}_student.csv")

    with open(std_path, "wb") as f:
        shutil.copyfileobj(standard_csv.file, f)

    with open(stu_path, "wb") as f:
        shutil.copyfileobj(student_csv.file, f)

    # 这里先写假结果，后面换成你的真实评分函数
    result = {
        "task_id": task_id,
        "student_id": "S001",
        "action_type": "basketball_shot",
        "total_score": 78.5,
        "rom_score": 82.0,
        "dtw_score": 75.0,
        "symmetry_score": 80.0,
        "rmse_score": 76.0,
        "latency_ms": round((time.time() - start_time) * 1000, 2)
    }

    return result

```

运行服务：

```
uvicorn main:app --reload --host 0.0.0.0 --port 8000
```

浏览器打开：

```
http://127.0.0.1:8000/docs
```

你会看到接口测试页面。

## 7.4 第三步：测试上传

用网页 `/docs` 测试，或者用 `curl`：

```
curl -X POST "http://127.0.0.1:8000/upload_motion" -F
"standard_csv=@../data/raw_csv/STD-basketball-shot-001.csv" -F
"student_csv=@../data/raw_csv/S001-basketball-shot-001.csv"
```

你应该得到类似结果：

```
{
  "task_id": "xxxx-xxxx-xxxx",
  "student_id": "S001",
  "total_score": 78.5,
  "latency_ms": 123.4
}
```

这里的 `task_id` 就对应案例里的：

CENI上传Task ID

如果接入真实 CENI，就把这个本地 `task_id` 换成 CENI 平台返回的任务编号。

## 7.5 第四步：记录端到端延迟

端到端延迟不是单纯算法时间，而是：

端到端延迟 = 上传时间 + 排队时间 + AI计算时间 + 结果回传时间 + Unity加载显示时间

可以在服务端记录：

```
upload_start = time.time()
# 接收文件
upload_end = time.time()

# 执行评分
score_start = time.time()
# scoring()
score_end = time.time()

# 返回结果
response_time = time.time()

log = {
    "upload_ms": (upload_end - upload_start) * 1000,
    "score_ms": (score_end - score_start) * 1000,
    "total_ms": (response_time - upload_start) * 1000
}
```

保存到：

```
logs/network/latency_log.csv
```

表格格式：

| task_id | upload_ms | score_ms | return_ms | total_ms |
|---------|-----------|----------|-----------|----------|
| xxx     | 80        | 250      | 60        | 390      |

验收目标：

正常状态：端到端反馈延迟  $\leq 500\text{ms}$   
故障恢复后：端到端反馈延迟  $\leq 800\text{ms}$

---

## 7.6 第五步：Wireshark抓包

你需要证明“数据确实经过网络传输”。

抓包步骤：

1. 打开 **Wireshark**。
2. 选择当前联网网卡。
3. 在过滤器输入：  
`tcp.port == 8000`
4. 启动抓包。
5. 上传CSV。
6. 停止抓包。
7. 保存为 **pcap** 文件。

文件保存：

```
logs/network/upload_test.pcap
```

报告里写：

本组使用**Wireshark**对**AI**评分服务上传过程进行抓包，过滤条件为**tcp.port==8000**，记录了**CSV**上传请求、服务端响应和结果回传过程。

---

## 8. 模块5：故障诊断具体怎么做

故障诊断最重要的是四个字：

有证据链

也就是说，你不能只说“我修好了”。你要证明：

故障前是什么样  
故障后出了什么问题  
我是怎么定位的  
我改了什么  
优化后改善了多少

---

## 9. 故障1：动捕标记点遮挡

### 9.1 怎么制造故障

方法1：真实遮挡。

让受试者做动作时，用身体或衣服遮住右腕/右肘标记点。

方法2：用CSV模拟遮挡。

直接把某些帧的右肘数据删掉。

```
import pandas as pd
import numpy as np

df = pd.read_csv("../data/raw_csv/S001-basketball-shot-001.csv")

# 假设右肘字段是 right_elbow_x/y/z
df.loc[100:130, ["right_elbow_x", "right_elbow_y", "right_elbow_z"]] =
np.nan

df.to_csv("../data/raw_csv/S001-basketball-shot-001-occlusion.csv",
index=False)
```

## 9.2 故障现象

1. CSV出现空值。
2. 右肘角度无法计算。
3. 角度曲线突然断裂。
4. Unity人物右臂抽动。
5. 评分明显下降或程序报错。

## 9.3 怎么定位

看三类证据：

1. FZMotion日志：是否提示marker lost。
2. Python缺失值统计：哪个关节、哪几帧缺失。
3. 角度曲线图：哪一段突然中断或跳变。

可以输出：

右肘在第100~130帧出现连续缺失，共31帧，占总帧数的3.1%。

## 9.4 怎么修复

小白优先用：

线性插值 + 高斯滤波

修复后重新计算：

缺失比例

RMSE

综合评分

## 9.5 报告怎么写

故障：右肘标记点遮挡。

现象：第100~130帧右肘坐标缺失，导致右肘角度曲线中断，综合评分由82.4下降至61.7。

定位：通过CSV缺失值统计发现`right_elbow_x/y/z`存在连续空值；FZMotion日志显示对应时间段 `marker lost`。

优化：对连续缺帧片段进行线性插值，并对角度曲线进行高斯滤波。

结果：优化后综合评分恢复至76.8，较故障状态评分偏差降低超过50%。

## 10. 故障2：T-Pose绑定偏差

### 10.1 怎么制造故障

真实方法：

让受试者T-Pose时手臂没有水平展开，或者身体略微歪斜。

模拟方法：

在Unity中使用错误Avatar绑定，或者在Python中给某一侧关节加固定偏移。

## 10.2 故障现象

1. 人物骨骼整体歪。
2. 左右肩高度不一致。
3. 左右对称性分数异常降低。
4. 某些关节长期偏差，而不是某几帧偶然偏差。

## 10.3 怎么定位

检查：

1. T-Pose截图。
2. 初始帧左右肩高度差。
3. 初始帧左右髋高度差。
4. 左右肢体角度是否长期存在固定偏差。

例如：

左肩初始高度  $y=1.52m$   
右肩初始高度  $y=1.43m$   
差值= $0.09m$   
说明T-Pose绑定可能存在偏差。

## 10.4 怎么修复

1. 重新做T-Pose。
2. 重新骨骼绑定。
3. 做骨骼尺度归一化。
4. 评分时先减去初始姿态偏移。

小白可以用“初始偏移校正”：

校正后角度 = 当前角度 - 初始帧角度偏移

## 10.5 报告怎么写

故障：T-Pose绑定偏差。

现象：学生动作在全程中右肩角度持续偏大，左右对称性得分由88.2下降至63.5。

定位：检查T-Pose截图发现右臂未完全水平展开；初始帧左右肩高度差为0.09m。

优化：重新进行T-Pose绑定，并在算法中加入初始姿态偏移校正。

结果：对称性得分恢复至82.7。

## 11. 故障3：网络延迟和丢包

### 11.1 怎么制造故障

如果是 Linux，可以用 `tc` 模拟：

```
sudo tc qdisc add dev eth0 root netem delay 300ms loss 1%
```

恢复网络：

```
sudo tc qdisc del dev eth0 root
```

如果是 Windows，小白可以用更简单的方法：

1. 在FastAPI接口里人为sleep 0.3秒。
2. 或者用网络限速/弱网工具。

FastAPI模拟延迟：

```
import time

@app.post("/upload_motion")
async def upload_motion(...):
    time.sleep(0.3) # 模拟300ms延迟
```

模拟随机丢包：

```
import random
from fastapi import HTTPException
```

```
if random.random() < 0.01:  
    raise HTTPException(status_code=503, detail="Simulated packet loss")
```

## 11.2 故障现象

1. 上传变慢。
2. JSON结果返回变慢。
3. Unity等待时间变长。
4. 端到端延迟超过500ms。
5. 严重时请求失败。

## 11.3 怎么定位

你要区分：

到底是网络慢，还是AI算法慢？

所以要记录：

上传耗时  
AI计算耗时  
回传耗时  
总耗时

如果：

AI计算时间没变，但总延迟变大

说明主要是网络问题。

如果：

AI计算时间变大

说明主要是算法或服务器问题。

## 11.4 怎么修复

常用策略：

1. 先传CSV，不传FBX。
2. CSV压缩后再传。
3. AI服务先返回JSON评分，PDF稍后生成。
4. Unity先显示文字评分，再加载动作动画。
5. 请求失败时自动重传。
6. 客户端增加缓存。

小白最容易实现的是：

先返回JSON，PDF和FBX后加载。

也就是把系统拆成：

实时反馈：JSON评分

非实时展示：PDF报告和FBX回放

## 11.5 报告怎么写

故障：CENI链路加入300ms延迟和1%随机丢包。

现象：端到端反馈延迟由420ms上升至930ms，超过实验设定的500ms阈值。

定位：通过服务端时间戳发现AI计算时间基本稳定在240ms左右，但上传与回传耗时明显增加；Wireshark抓包显示存在重传包。

优化：采用CSV优先上传、JSON优先回传、PDF延迟生成策略，并在客户端加入一次重传机制。

结果：优化后端到端反馈延迟下降至760ms，满足故障恢复后≤800ms要求。

## 12. 故障4：AI评分漂移

### 12.1 怎么制造故障

方法1：人为修改某个关节角度。

```
df["right_elbow_x"] = df["right_elbow_x"] + 0.2
```

方法2：修改权重。

把右肘权重从30%错误改成80%。

方法3：关键帧选错。

把出手帧选成准备帧。

---

## 12.2 故障现象

1. 同一个动作多次评分差异很大。
2. 某个关节误差异常大。
3. 总分和肉眼观察不一致。
4. 单个指标支配最终结果。

---

## 12.3 怎么定位

检查四个地方：

1. 预处理是否一致。
2. 标准动作模板是否固定。
3. 关键帧是否选对。
4. 权重是否设置合理。

你要输出中间指标，不要只输出总分。

错误示例：

总分：55

正确示例：

ROM：82  
DTW：78  
对称性：80  
RMSE：35

总分：62

最大异常：右肘RMSE = 31.5°

这样你能定位是 RMSE 指标出了问题。

## 12.4 怎么修复

1. 固定标准动作模板。
2. 固定归一化方法。
3. 限制单个指标最大扣分。
4. 剔除异常帧。
5. 检查权重配置文件。

例如可以限制：

单个关节误差不能让总分一次性扣超过20分。

## 13. 故障5：Unity帧同步错位

### 13.1 怎么制造故障

让标准动作从第0帧开始播放。  
让学生动作从第10帧开始播放。

或者：

两个动画播放速度不同。

### 13.2 故障现象

1. 两个人动作明显不同步。
2. 本来相似的动作被误判为偏差大。
3. 高亮关节出现时机不对。
4. 关键帧截图不对应。

## 13.3 怎么定位

需要记录：

标准动作当前帧  
学生动作当前帧  
当前播放时间  
动画速度

例如：

标准动作：frame 125  
学生动作：frame 137  
帧差：12帧

实验要求：

双通道回放帧同步误差  $\leq 2$  帧

所以 12 帧明显不合格。

## 13.4 怎么修复

1. 播放前统一从第0帧开始。
2. 两个动画设置相同播放速度。
3. 根据时间戳同步，而不是只靠Unity默认播放。
4. 如果动作长度不同，先用DTW或重采样对齐。

小白版先做：

两个动画同时触发播放  
播放速度都设为1.0  
播放前都归零

## 14. 每天具体该做什么

## 第1天

1. 建项目文件夹。
2. 安装Python、VS Code、Unity、Wireshark、Git。
3. 找老师/师兄要一组CSV和FBX。
4. 用Python成功读取CSV。

目标：

能打印CSV前5行。

---

## 第2天

1. 写数据字典。
2. 检查CSV缺失值。
3. 画一个右肘角度曲线。

目标：

生成 `right_elbow_angle_compare.png`。

---

## 第3天

1. 做缺帧插值。
2. 做高斯滤波。
3. 计算右肘ROM和RMSE。

目标：

输出一个简单评分。

---

## 第4天

1. 扩展到肩、肘、腕、膝几个关节。
2. 计算ROM、DTW、对称性、RMSE。
3. 输出JSON结果。

目标：

生成 `result.json`。

---

## 第5天

1. 生成PDF反馈报告。
2. 把FBX导入Unity。
3. 让标准动作和学生动作能播放。

目标：

Unity里能看到两个人物动作。

---

## 第6天

1. Unity读取JSON。
2. 显示总分。
3. 做出错关节高亮。

目标：

右肘偏差 $>15^{\circ}$ 时能高亮。

---

## 第7天

1. 写FastAPI上传接口。
2. 上传CSV。

3. 返回评分JSON。
4. Wireshark抓包。

目标：

跑通网络版评分。

## 15. 最小可行版本验收标准

你最少要做到下面这些：

1. 有一组标准动作CSV和学生动作CSV。
2. Python能读取CSV。
3. Python能计算至少3个关节的角度误差。
4. 能输出一个0-100分。
5. 能生成JSON结果。
6. 能生成简单PDF报告。
7. Unity能播放标准动作和学生动作FBX。
8. Unity能显示评分。
9. FastAPI能接收CSV并返回JSON。
10. 能模拟一个故障，并给出优化前后对比。

只要这 10 个做完，项目就有骨架了。

## 16. 汇报时可以这样说

本项目首先基于FZMotion完成体育动作采集，导出CSV关键点时序数据和FBX骨骼动画文件。然后，利用Python对CSV数据进行缺帧插补、坐标归一化和高斯滤波，并计算ROM、DTW、左右对称性和关键关节角度RMSE四类指标，形成0-100分综合评分。接着，在Unity中导入FBX文件，实现标准动作与学生动作的双通道同步回放，并根据关节角度偏差对错误关节进行高亮。最后，通过FastAPI/CENI完成动作数据上传、AI评分服务调用和结果回传，并在标记点遮挡、网络延迟、随机丢包和评分漂移场景下进行故障定位与优化。

## 17. 你现在最该先做哪一步

现在先不要管完整系统。

第一步：

拿到一组标准动作CSV和一组学生动作CSV。

第二步：

用Python读出来，打印前5行。

第三步：

画出一个关节角度曲线，比如右肘。

只要能画出这张图，后面的 ROM、DTW、RMSE、PDF报告、Unity高亮，本质上都是在这张图的基础上继续扩展。

---

## 18. 总结

这个项目的核心不是“做一个很厉害的AI模型”，而是完成一个可验证的系统闭环：

采集有数据  
分析有指标  
仿真有画面  
传输有日志  
故障有证据  
优化有对比

最重要的一句话：

**先跑通闭环，再追求指标。**